Information System Design: Revision Guide

ISDesign 2026/2027, Dr. Ameni Youssfi Nouira, PRISTINI (GLSI2 / BI2). Built from: Chapter 1 (Introduction, 14 slides), Chapter 2 (Use Case, 12 slides), Tutorial 1 (Use Case), Tutorial 2 (Class Diagram), Tutorial 2.1 (Use Case & Class Diagram) and the 5 photos of the board corrections.

Contents: 0. What she asks and how the exam works · 1. Chapter 1: IS, development steps, design, modeling, UML · 2. Chapter 2: the use case diagram · 3. Method: how to solve a use case exercise · 4. Tutorial 1 solved (with board photos) · 5. Class diagrams (no slides yet, explained from zero) · 6. Tutorials 2 and 2.1 solved · 7. Exam-day checklist

Labels used: **(course)** = the slide's words · **(board)** = what the prof drew in class · **(extra)** = standard UML that isn't on the slides but that you need for the TDs. All the clean diagrams were redrawn by me from the board photos and the TD texts.

0. What she asks and how to win the marks

🎯 **What the tutorials tell us about the exam**

Every tutorial is built the same way: **a short text describing a company/situation → "Draw the use case diagram" and/or "Draw the class diagram"**. Variants she already used:

- **Draw the use case diagram** from a text (TD1 Ex1, Ex2, TD2 Ex1, Tutorial 2.1, Ch2 activity). This is the core skill: expect at least one.
- **Critique a student's diagram** and list the "pitfalls", then propose a correct one (TD1 Ex3). You must know the classic mistakes (§3.4).
- **Define the actors** of the system (TD1 Ex3 2.1).
- **Draw the class diagram** from a text (TD2 Ex1, Ex2, Ex3, Tutorial 2.1).
- **Draw an object diagram** matching a text (TD2 Ex3 Q2).
- **Course questions** on Chapter 1: definitions (IS, its roles, model, method vs language), development steps, the 3 analysis axes, UML history (3 amigos, OMG 1997), types of UML diagrams.

1. Chapter 1: Introduction (slides 1–14)

1.1 What is an Information System (IS)? (slide 3)

Definition (course: learn it word for word)

An information system represents **the set of elements involved in the management, storage, processing, transmission and dissemination of information within an organization**.

Its 4 roles (slide 3):

1. **Collection** of information (gathering it),
2. **Storage** (keeping it, e.g. a database),
3. **Processing** (computing, transforming it),

Simple example: your school's IS

- *Collection*: students fill in the registration form.
- *Storage*: students and grades saved in the database.

4. **Dissemination** (sending it to the people who need it).

- *Processing*: averages computed automatically.
- *Dissemination*: transcripts sent / posted on the platform.

An IS is not only computers: people, procedures and data are part of it too. The computer part is the "computerized system" we model.

1.2 Steps in the development of an IS (slides 4–6)

Slide 4 (diagram): 8 steps

```
1. Needs analysis
2. Feasibility study
3. System design
4. Development and implementation
5. Testing and validation
6. Deployment
7. Training and support
8. Maintenance and evolution
```

Memory trick: "**N**ever **F**orget **D**esign, **D**evelop, **T**est, **D**eploy, **T**rain, **M**aintain".

Slides 5–6 (text): detailed steps

Step	What it means
Requirements expression	write the **requirements specification document** (cahier des charges)
Specification	**what** the system should be and **how** it can be used
Analysis	the elements of the IS, their structures and relationships, along **3 axes** (below)
Design	**technical solutions**: architecture, performance, optimization, structures and algorithms
Implementation	development and programming
Testing & verification	quality controls **throughout** the development cycle
Validation	check compliance with the requirements specification + discussion with the user
Maintenance & evolution	**corrective** (fixing bugs) and **adaptive/evolutionary** (integrating new changes)

🎯 The 3 axes of analysis (slide 5): very likely question

Axis	Course words	Simple meaning	UML diagram that shows it (extra)
Functional	object know-how	WHAT the system does (its functions)	**Use case** diagram
Static	object structure	WHAT it is made of (data, links), doesn't move in time	**Class** / object diagram
Dynamic	object life cycle	HOW it behaves over time (states, messages)	Sequence, state machine, activity

That's why the course goes in this order: Use case (functional) → Class diagram (static) → dynamic diagrams later.

1.3 Design (slide 7)

- A **creative process** for developing software.
- Gives the software an **architecture** by dividing it into **modules**, each responsible for different functionalities.

Architectural design	Detailed design
• internal structure of the software • decomposition into manageable-sized components • interfaces and interactions between components	• role of each component • its subcomponents

Analogy: architectural = the plan of the floors and rooms of a house; detailed = the plumbing and wiring inside each room.

1.4 Modeling, model, method, language (slides 8–9)

Modeling = support for design: **formalizing** the solution with known notations/rules. It **reduces complexity**: remove non-significant details, keep what matters to understand and predict.

Model (course)
An **abstract and simplified representation** of a real-world entity, aimed at **describing, explaining or predicting** it. Example: a **plan** (of a building).

	Design method	Modeling language
Definition	normative description of the **steps** of modeling (what to do, in what order)	a **graphical language** to represent and communicate the aspects of an IS: **vocabulary + rules**
Example	Merise	UML
Problem	many specific cases → hard to make an exhaustive method	-

⚠ **Careful**
Slide 9 lists "Merise, UML" as examples of design methods, but **UML is a modeling language, not a method** (the "L" = Language). It says how to *draw*, not which steps to follow. If asked, say: "UML is a modeling language; Merise is a method."

1.5 UML: history and building blocks (slides 10–14)

```
1980s   Merise: functional programming methods,
        data SEPARATED from processes
1990–95 object-oriented programming appears
        → more than 50 methods!
1994    consensus on 3 methods:
        OMT  (James Rumbaugh): static, dynamic and
              functional aspects, graphically
        OOD  (Grady Booch): package concept
        OOSE (Ivar Jacobson): user requirements
              (→ he invented use cases)
        the "3 Amigos" unite → UML
1997    UML = standard of the OMG
        (Object Management Group)
```

UML = Unified Modeling Language. Used in information systems, telecoms, transport, aeronautics, aerospace, science, and even to model mechanical or human behaviours.

3 basic building blocks (slide 13):

- **Elements**: the essential abstractions of the model (actor, class, use case…).
- **Relationships**: the links between elements (association, include, inheritance…).
- **Diagrams**: visual representations of the elements; each gives a **partial view** from one perspective (user, admin…). **All diagrams together** give the overall view of the system.

UML diagram types (slide 14): 14 diagrams in 2 families

Structural (static) = 7	Behavioral (dynamic) = 7
Class · Object · Component · Composite structure · Package · Deployment · Profile	Use case · Activity · State machine · **Interaction** (4): Sequence · Communication · Interaction overview · Timing

Trap: the **use case** diagram is **behavioral**, not structural. Class and object diagrams are structural.

1.6 Chapter 1: questions de cours with model answers

Question	Answer
Define an IS and give its roles.	Set of elements involved in managing, storing, processing, transmitting and disseminating information in an organization. Roles: collection, storage, processing, dissemination.
List the steps of IS development.	Needs analysis → feasibility study → design → development/implementation → testing/validation → deployment → training/support → maintenance/evolution.
What are the 3 axes of analysis?	Functional (object know-how), static (object structure), dynamic (object life cycle).
Corrective vs adaptive maintenance?	Corrective = fixing bugs; adaptive/evolutionary = integrating new changes.
Validation vs testing?	Testing = quality controls during the whole cycle; validation = checking the product matches the requirements specification, with the user.
Architectural vs detailed design?	Architectural: internal structure, decomposition into components, their interfaces/interactions. Detailed: role of each component and its subcomponents.
What is a model? Why model?	An abstract, simplified representation of a real entity to describe, explain or predict it. Modeling reduces complexity by removing non-significant details.
Method vs modeling language?	Method = normative steps of modeling (Merise); language = graphical vocabulary + rules to represent/communicate (UML).
Where does UML come from?	From unifying OMT (Rumbaugh), OOD (Booch) and OOSE (Jacobson), the "3 Amigos"; OMG standard in 1997.

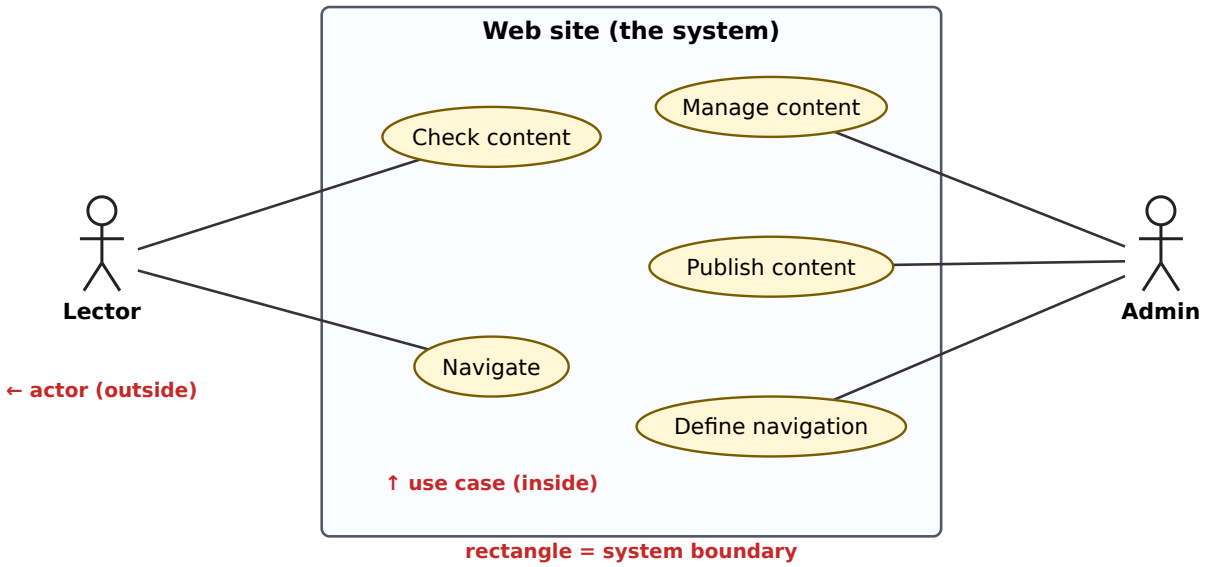
Building blocks of UML?	Elements, relationships, diagrams.
Why several diagrams?	Each diagram is a partial view of one aspect; together they give the overall view of the system.

2. Chapter 2: the Use Case diagram (slides 1–12)

2.1 What it is (slide 3)

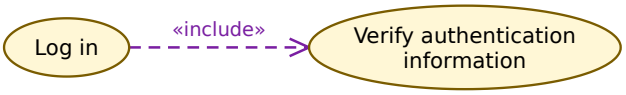
- **The functional UML diagram** (the functional axis of §1.2).
- A way to specify the **uses / features** of a system: **what** the system does for its users, **not how**.
- Represents the **interactions between users and the system**.
- A **graphical representation** (the diagram) **+ a textual description** (the documentation, §2.6).

2.2 The basic elements (slides 4–5)

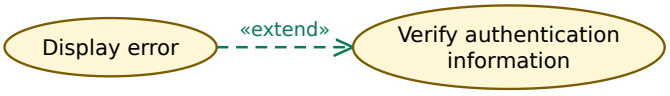


Element	Course definition	How to recognise it / write it
Actor (stick figure)	Role played by any **external** entity that interacts with the system.	• It's a **role**, not a person: Ahmed can be a "Student" and a "Customer". 5 employees with the same job = **one** actor "Employee". • It can be a **human** or **another system** (e.g. "Personnel management system", "Bank"). Put it **outside** the rectangle. • (extra) **Primary** actor = starts the use case to get a goal (usually on the left); **secondary** actor = is asked for help by the system (usually on the right).
Use case (ellipse)	Specification of a set of actions performed by the system.	• Name = **verb + complement**, from the actor's point of view: "Borrow a book", "Reserve a room". • A **goal** that gives a result of value to the actor, not a button click or a technical step.
System (rectangle)	The boundary of the system (title = system name).	Use cases inside, actors outside.

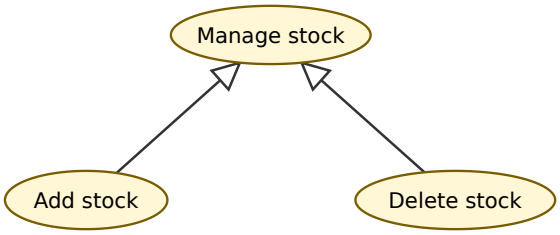
2.3 The 4 relationships (slides 4–7): the heart of the chapter



INCLUDE: Log in ALWAYS does Verify.
Arrow: base UC → included UC



EXTEND: Display error happens SOMETIMES (only if the info is wrong).
Arrow: optional UC → base UC



Generalization of UCs: Add / Delete are kinds of Manage



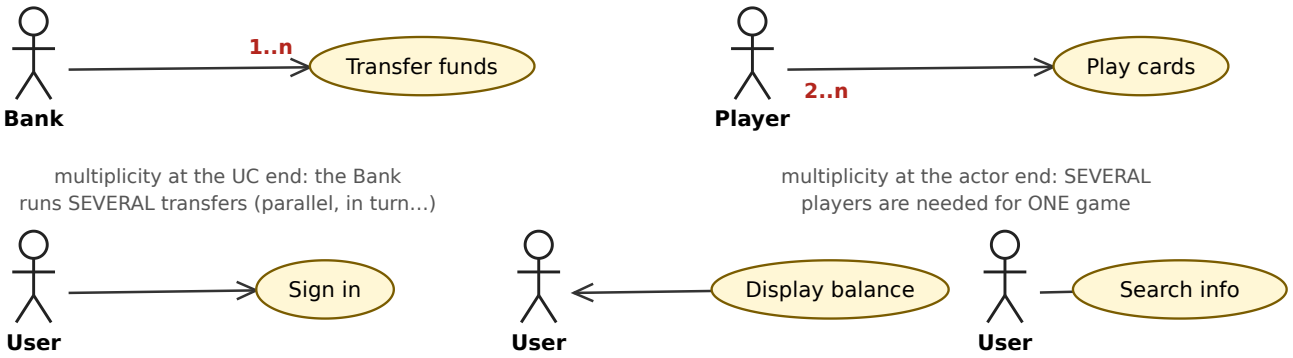
Admin is a kind of User (does all User does + more)

Relationship	Course definition	Notation & arrow direction	Key words in the text
Interaction (association)	Relationship between an actor and a UC.	Plain solid line (optional arrow, §2.5).	"The member can borrow..."
Include	A UC **contains** the actions defined in another UC.	Dashed arrow `«include»` from the **base UC → the included UC**. The included one is **always** executed.	"must be preceded by", "requires", "always", "each time... the system checks"
Extend	A UC **can be extended** by another UC.	Dashed arrow `«extend»` from the **optional UC → the base UC**. The extension happens **only sometimes** (a condition).	"optionally", "may", "can also", "in case of", "if..."
Generalization	A UC is a **specification** of another UC; an actor is a specification of another actor.	Solid line + **hollow triangle** pointing to the **general** one (child → parent).	"there are 2 kinds of...", "can be X or Y", "an engineer is an employee who can also..."

💡 **Include vs extend: the trick to never confuse the arrows**
Include: read the arrow as "*needs*": "Log in **needs** Verify authentication". The arrow leaves the one who needs.
Extend: read the arrow as "*can be added to*": "Display error **can be added to** Verify authentication". The arrow leaves the optional one.
In both cases the arrow points **towards the main/base story** for extend, and **towards the reusable piece** for include.

⚠ **Actor generalization (very used on the board)**
If the child actor (Engineer, Customer, Head teacher) can do **everything** the parent does (Employee, Visitor, Teacher) **plus more** → triangle from child to parent. The child then inherits all the parent's use cases: **don't** draw the parent's lines again from the child.

2.4 Multiplicities (refinement concepts 1–2, slides 8–9)



- **At the UC end** (e.g. `1..n` near "Transfer funds"): the actor is involved in **several instances** of this UC: in parallel, by mutual exclusion, or at different times (UML doesn't say which).
- **At the actor end** (e.g. `2..n` near "Player"): **several instances of the actor** are needed to perform **one** UC: simultaneous actions, or complementary and successive actions.

2.5 Interaction orientation (refinement concept 3, slide 10)

An arrow on the actor–UC line shows the **direction of the interaction** (who starts / who receives): "User → Sign in" (the user triggers it), "Display balance → User" (the system sends to the user). **No arrow = bidirectional.** In the board corrections, the prof often puts the arrow from the actor to the UC.

2.6 Use case documentation (textual description, slide 11)

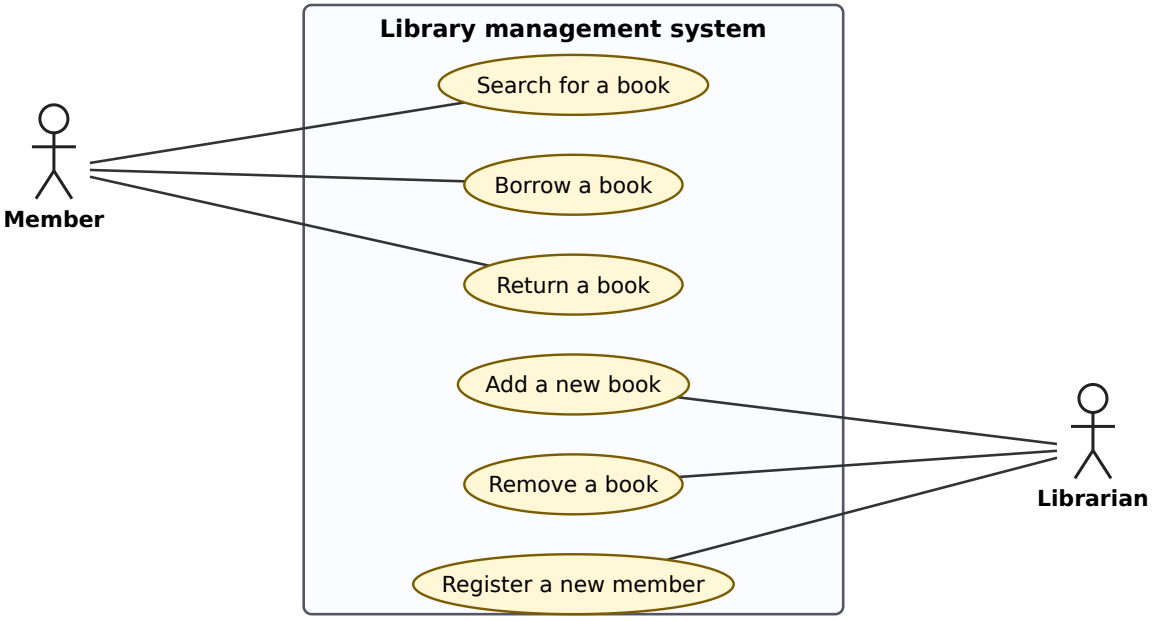
The template to fill (learn the headings in order):

```
Use Case Name:
Actors involved:
Description:     // brief description of the
UC
Preconditions:   // to verify BEFORE the UC
begins
Main flow / Basic sequence:
   1. …
   2. …
Alternative flows / Exceptions:
   If … then …
Postconditions:  // to verify AFTER the UC
ends
```

Filled example: "Borrow a book"

```
Name: Borrow a book
Actors: Member (primary), Librarian
Description: a member borrows an available
copy.
Preconditions: the member is registered and
   authenticated; the book exists.
Main flow:
  1. The member searches for the book.
  2. The system shows the available copies.
  3. The member chooses a copy.
  4. The system records the loan + return
date.
Alternative flows:
  If no copy is available → the system
proposes
     a reservation.
  If the member has 3 loans → refusal message.
Postconditions: the loan is recorded; the copy
   is marked "borrowed".
```

2.7 Slide 12 activity: the library (solved)



Two actors, six use cases, each linked to the actor who performs it. A good bonus: "Borrow a book" `«include»` "Search for a book" is **not** required by the text, so don't invent relationships the text doesn't give.

3. Method: solving a use case exercise in the exam

3.1 The 7 steps (do them in this order, on the draft first)

1. **Read the whole text once** without writing. Find the name of the **system** (it becomes the rectangle title).
2. **Underline the actors**: the nouns of **people / roles / external systems** that *do* something or are *contacted* (employee, manager, engineer, bank, "personnel management system"). Ignore things the system manages (books, products: those are *data*, future classes).
3. **Circle the actions**: the verbs "the X can/must…" → each becomes a use case named **verb + complement**.
4. **Make a table actor → use cases** (2 columns). This is the skeleton; it already earns most marks.
5. **Find the relationships** with the key words (table §3.2): include, extend, generalization of actors, generalization of UCs.
6. **Draw**: rectangle with the title; primary actors on the left, secondary actors on the right; group the UCs of each actor near it; avoid crossing lines.
7. **Check** with the list of mistakes (§3.4): every requirement of the text is covered, no arrow between UCs without a stereotype, arrows in the right direction.

3.2 Dictionary: text → UML (built from the sentences of her TDs)

Sentence in the text	What to draw	Where she used it
"All consultations must be preceded by a light authentication"	Consult `«include»` → Light authentication	TD1 Ex2
"Only teachers can reserve… subject to the availability"	Reserve room `«include»` → Check availability	Tutorial 2.1

"may optionally be accompanied by printing"	Print documents `«extend»` → Consult / Update	TD1 Ex2
"This employee may optionally consult the catalog"	Consult catalog `«extend»` → Submit request	TD1 Ex1
"If the copy is not returned within 3 weeks, it generates a dispute"	Manage disputes `«extend»` → Return a work [late]	TD1 Ex3
"All personnel can consult… Engineers can (also) update"	Actor Engineer —▷ Employee	TD1 Ex2
"a designated teacher who alone is allowed to…"	Actor Head teacher —▷ Teacher + its own UC	Tutorial 2.1
"either to check if a product exists or to freely browse"	Check / Browse —▷ Consult products (UC generalization)	TD1 Ex2
"update operations: adding, removing and modifying"	Add / Remove / Modify —▷ Update products	TD1 Ex2, Ex3
"a password verified by contacting the personnel management system"	Secondary actor (a system) linked to Strong authentication	TD1 Ex2
"All operations are recorded in an access log"	Every operation `«include»` → Save in access log	TD1 Ex2
"Access to the system must be protected by a password"	a "Log in / Authenticate" UC (or a precondition)	TD1 Ex3
"only one employee now, but more in the future"	still **one** actor "Employee" (an actor is a role)	TD1 Ex3

3.3 How to lay out the drawing so it's readable

- Draw the **rectangle big** (whole page width). Write the system name at the top.
- Put the actor who has the most UCs on the left middle; put the UCs that only he uses in a column next to him.
- Put the **included / extending** UCs further right, so dashed arrows go left→right and don't cross the actor lines.
- Put the children of a generalization in a small column next to their parent (like "Manage members" → Add / Update / Delete on the board).
- Write `«include»` / `«extend»` on **every** dashed arrow, and the condition in brackets if the text gives one: `«extend» [late > 3 weeks]`.

3.4 The classic mistakes (= the answer to "critique this diagram")

Pitfalls to look for (and to avoid yourself)

1. **No system boundary** (rectangle missing).
2. **Sequencing / flowchart**: arrows between UCs to show an order ("log in → search → check out"). A use case diagram shows **no order and no flow**; that's for activity/sequence diagrams.
3. **Arrows between UCs without a stereotype**: every UC–UC link must be `«include»`, `«extend»` or a generalization.
4. **Too fine granularity / functional decomposition**: one UC per tiny operation (add member, modify member, delete member each tied to the actor) instead of one "Manage members" (possibly with children).

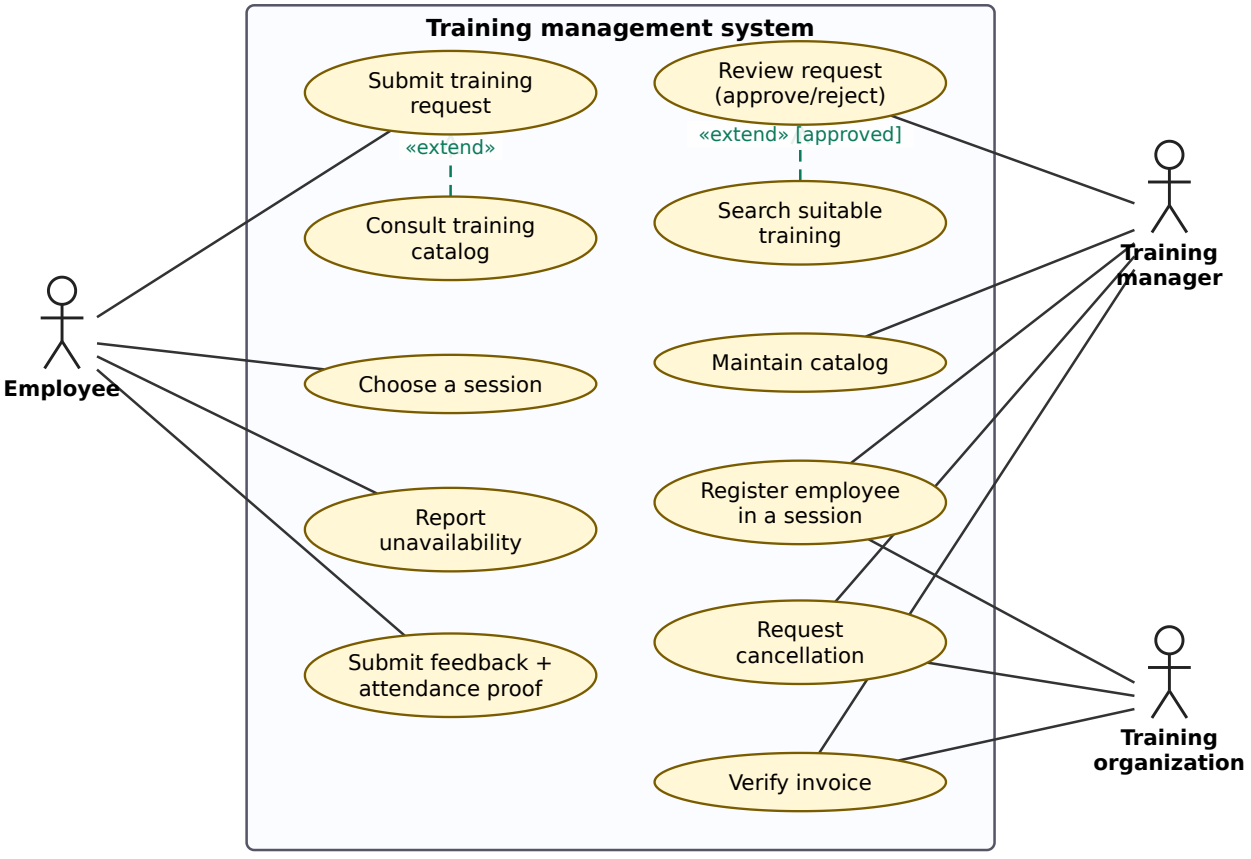
5. **Technical steps as use cases**: "Log in", "Click OK", "Save to database" are not goals by themselves (authentication = a precondition or an included UC).
6. **Missing requirements**: a function of the text has no UC (e.g. legal proceedings, loans).
7. **Wrong actors**: data shown as actors (a book), or a person instead of a role, or the system itself as an actor.
8. **Wrong arrow direction** for include/extend, or confusing them.
9. **Bad names**: no verb, mixed languages ("return oeuvre"), inconsistent style.
10. **Crossing, tangled lines** that make it impossible to see which actor does what.

4. Tutorial 1 (Use Case) solved

Exercise 1: Employee training process

Step 4 table (actors → use cases)

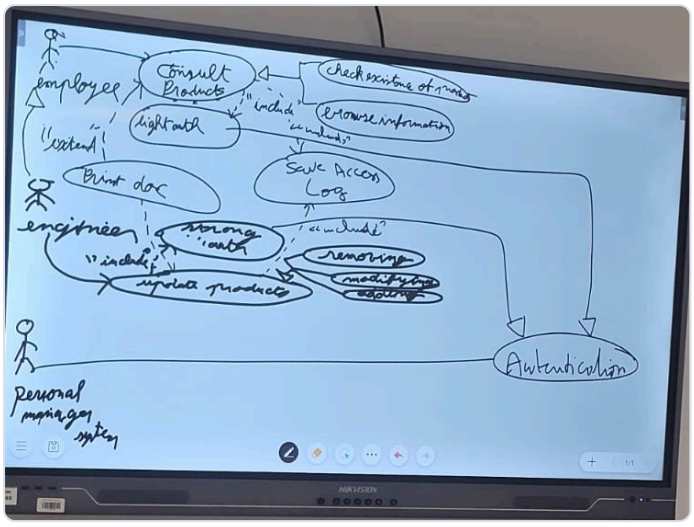
Actor	Use cases (from the text)
Employee (primary)	Submit a training request · (optionally) consult the catalog · choose a session · report that he can't attend · submit feedback + attendance certificate
Training manager (primary)	Review the request (approve/reject) · search a suitable training (only if approved) · maintain the catalogs · register the employee · request cancellation · verify the invoice
Training organization (secondary, external)	receives the registration and the cancellation, sends the invoice



Key choices: "may optionally consult the catalog" → `«extend»` ; "In case of approval, the manager searches" → `«extend» [approved]` ; the training organization is an external actor because the registration and the invoice go

to/come from it. (My proposed solution: there's no board photo for this exercise.)

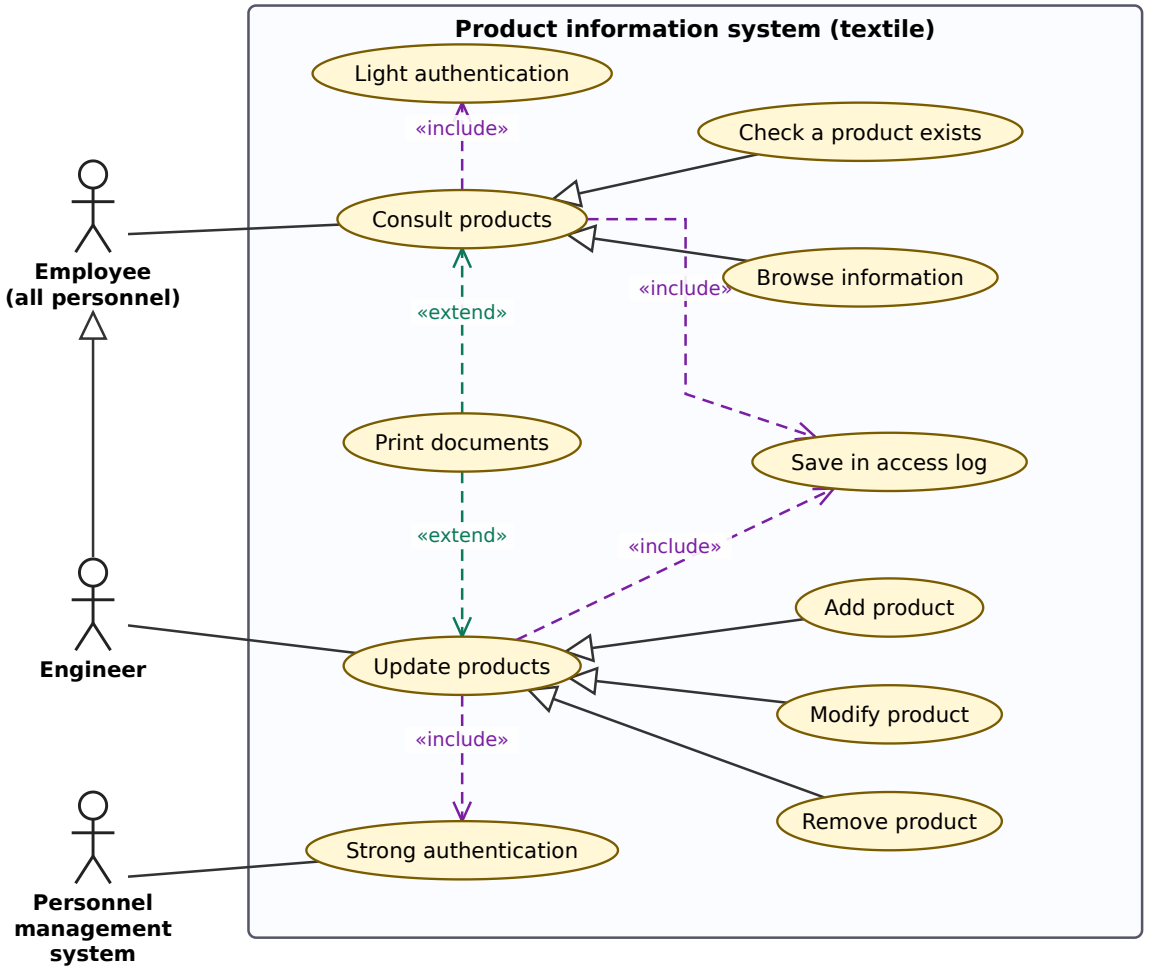
Exercise 2: Textile company: product information



(board) the prof's correction

How the prof read the text:

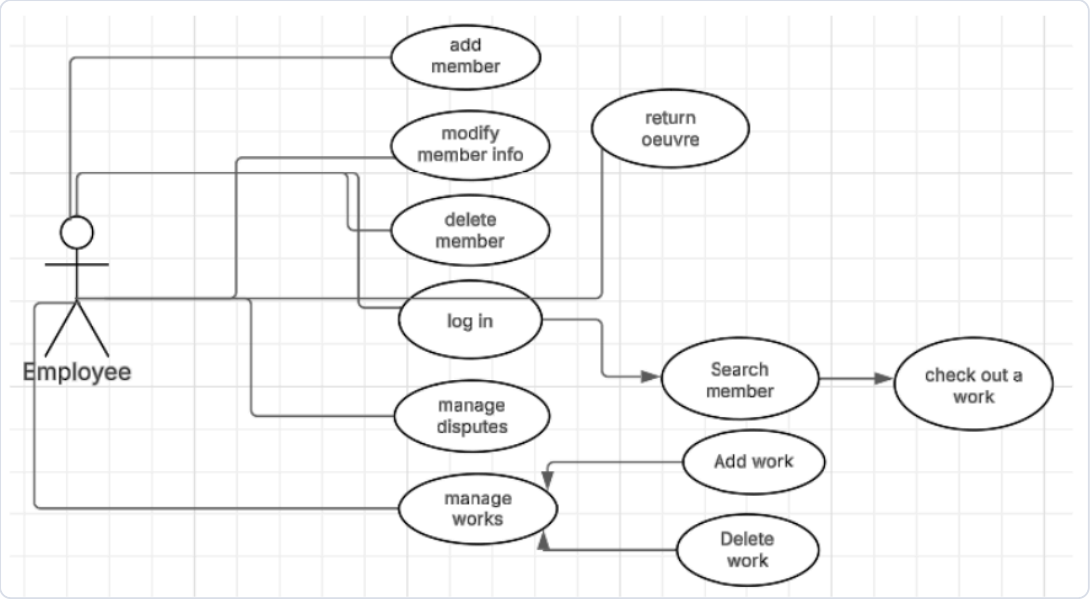
- "All personnel can consult" → actor **Employee**; "Engineers can perform update operations" → actor **Engineer** —▷ **Employee** (an engineer can also consult).
- "check if a product exists or freely browse" → 2 kinds of Consult (generalization).
- "preceded by a light authentication" → `«include»`; "more thorough authentication" for updates → `«include»` Strong auth.
- "password verified by contacting the personnel management system" → **secondary actor** (a system).
- "recorded in an access log" → `«include»` Save access log, from Consult and from Update.
- "may optionally be accompanied by printing" → Print documents `«extend»`.



Small improvement vs the board

On the board, both authentications are children of one "Authentication" UC linked to the personnel system. But only the **strong** one contacts the personnel system (the light one just asks name + department). So link the system to **Strong authentication** only. Both versions show the same understanding; mention this if you have time.

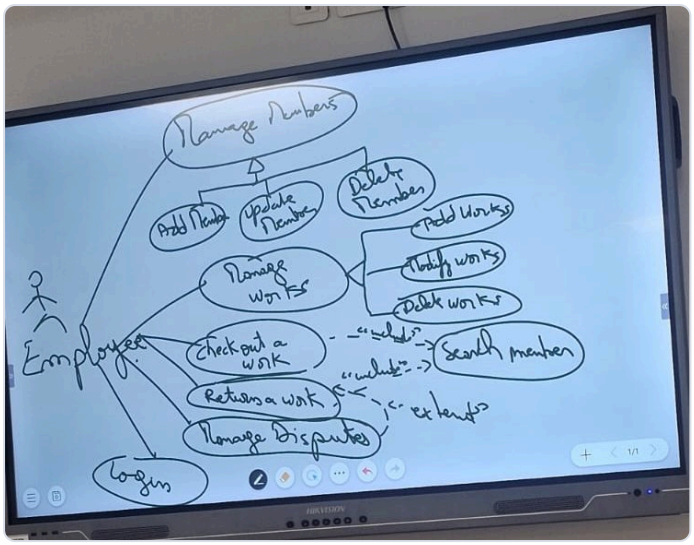
Exercise 3: Media library: critique and correct



The student's diagram (from the TD sheet)

Q1: Critique: the pitfalls this student fell into

1. **No system boundary** (no rectangle "Media library system").
2. **Sequencing**: "log in → Search member → check out a work" are linked by plain arrows to show an **order of steps**. A use case diagram doesn't model order/flow.
3. **Links between UCs without stereotype**: Add work / Delete work → manage works, log in → search member… plain arrows mean nothing in UML. It should be `«include»`, `«extend»` or generalization.
4. **Too fine granularity and inconsistency**: add / modify / delete member are 3 separate UCs each linked to the actor, while for works there is a "manage works". Group them into "Manage members" (with Add/Update/Delete as children).
5. **"Log in" used as a step of a chain**: authentication is a precondition (or a separate UC), not the start of a flowchart.
6. **Missing requirements**: legal proceedings (after 1 year) are missing; the link between a late return (3 weeks) and the dispute isn't shown; loans aren't modeled as a whole (no "return" linked properly).
7. **Wrong direction**: "check out a work" needs to search the member, so it should be Check out `«include»` Search member (arrow from check out to search), not search → check out.
8. **Names**: "return oeuvre" mixes French and English; names must be consistent verb + complement.
9. **Tangled lines**: crossing connectors make it impossible to see which UC belongs to the actor.

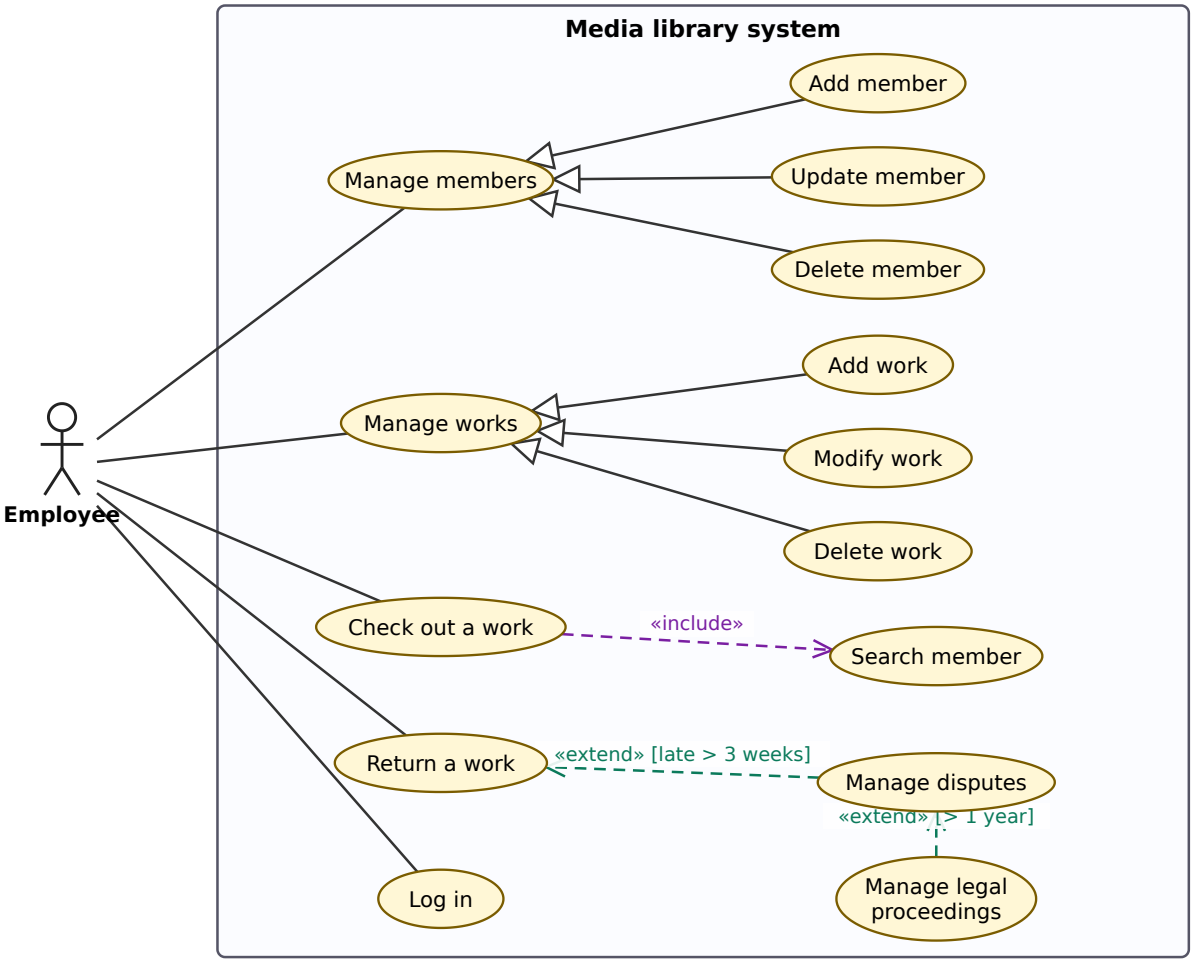


(board) the prof's corrected diagram

Q2.1: Actors of the computerized system: Employee only. Even if more employees are hired later, they play the **same role** → still one actor. (Members don't use the software themselves; the employee does everything for them.)

Q2.2: what the board shows: Manage members (children Add / Update / Delete), Manage works (children Add / Modify / Delete), Check out a work `«include»` Search member, Return a work, Manage disputes `«extend»` Return a work, Login.

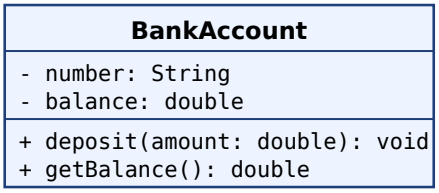
In my clean version below I also added **Manage legal proceedings** `«extend»` Manage disputes [> 1 year], because the text asks for it.



5. Class diagrams, from zero (extra: no slides yet)

You weren't there and there's no chapter PDF in the repo, but Tutorials 2 and 2.1 ask for class diagrams, so here is everything you need. Good news: **it's your Java course drawn as boxes**.

5.1 A class box



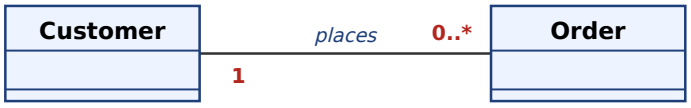
- Top: the **class name** (capital letter, singular: `Book`, not `books`).
- Middle: **attributes** `visibility name: Type`.
- Bottom: **operations** (methods) `visibility name(param: Type): ReturnType`.

```java
public class BankAccount {
    private String number;
    private double balance;
    public void deposit(double amount) { … }
    public double getBalance() { … }
}
```

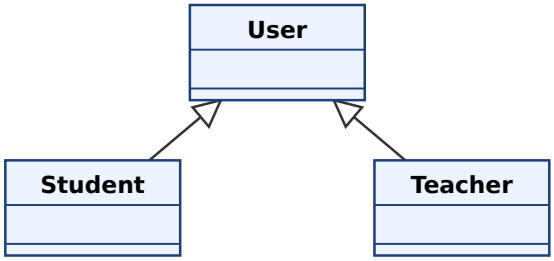
UML	Java	Meaning
`+`	public	everyone
`-`	private	only the class
`#`	protected	package + subclasses
`~`	(none)	package

Rule of thumb: attributes `-` (encapsulation), operations `+`. Types: `String` for text (not `char` as on the board), `Date` for dates (not `int`).

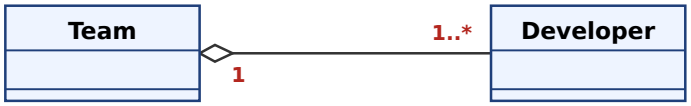
5.2 Relationships between classes



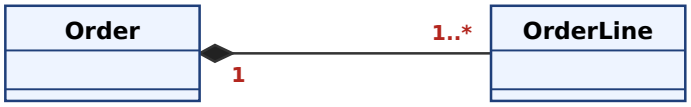
ASSOCIATION: a customer places 0 or many orders; an order is placed by exactly 1 customer.



GENERALIZATION (inheritance, "is a"):
a Student IS A User → inherits id, name…
Java: class Student extends User



AGGREGATION (empty ◇, "has"): the developer can exist without the team (he can change team).



COMPOSITION (filled ◆, "made of"): an order line can't exist without its order (delete order → lines die).

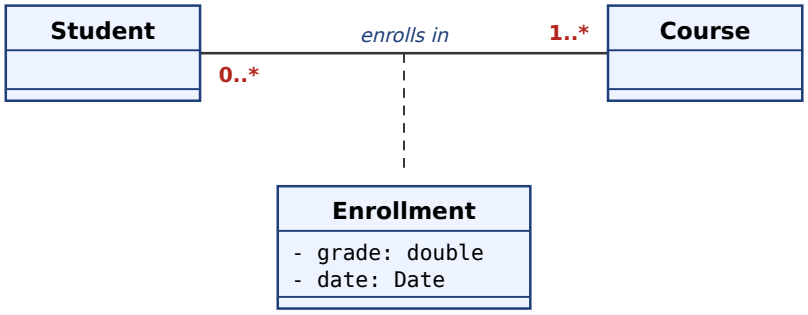
Multiplicities (written at the end of the line, near the class they count):

Write	Means
`1`	exactly one
`0..1`	zero or one (optional)
`*` or `0..*`	zero or many
`1..*`	at least one
`2..5`	between 2 and 5

The board also uses `1` / `N` (Merise style). In UML, write `*` or `1..*`.

> 💡 **How to find a multiplicity: ask the question twice**
> "**One** Customer places **how many** Orders?" → 0 or many → write `0..*` **next to Order**.
> "**One** Order is placed by **how many** Customers?" → exactly 1 → write `1` **next to Customer**.
> The number always goes on the side of the class you're **counting**.

Association name: a verb in the middle of the line ("places", "makes", "works for").



ASSOCIATION CLASS: the grade belongs neither to the student alone nor to the course alone, but to the PAIR (student, course Typical with many-to-many (* — *).

5.3 Object diagram

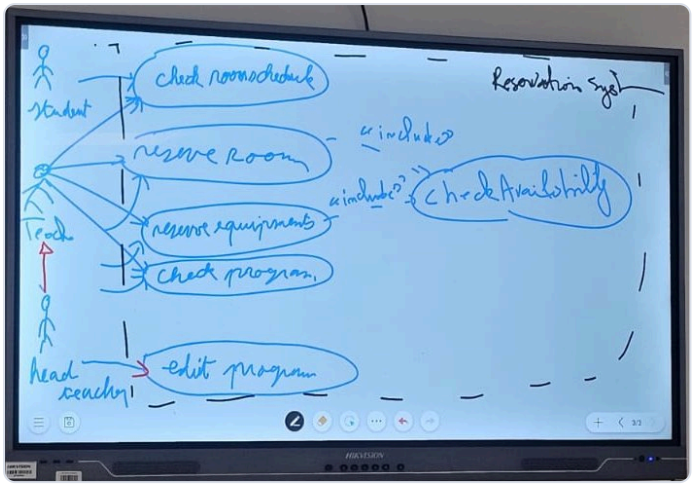
A **snapshot** of real objects at one moment, following a class diagram: box title objectName : ClassName (underlined), attributes with **values** (`name = "Ali"`), lines = **links** (instances of associations), **no multiplicities**, no operations. Example: TD2 Ex3 Q2 (§6.4).

5.4 Method for a class diagram exercise

1. **Nouns → classes or attributes.** A noun with its own information (Teacher, Reservation, Room) = a class. A simple value (name, date, capacity, price) = an attribute.
2. **"is a / can be X or Y / there are 2 kinds of"** → generalization (Programmer and Designer are Developers; Student and Teacher are Users).
3. **Verbs between two classes → associations** ("a teacher *makes* reservations", "a team *works for* a company").
4. **"is composed of / is made of / contains"** → composition (the part dies with the whole: board → cells) or aggregation (the part can live alone: team → developers).
5. **Multiplicities**: ask the question twice for every association (§5.2). Numbers in the text ("three wheels", "8 neighbours") become multiplicities.
6. **Many-to-many with data** (a grade, a quantity, a date of the pair) → association class.
7. **Operations**: the verbs the class itself does (uncover(), mark(), compute()). The text's rules become operations.
8. **Check**: every noun of the text is somewhere; no attribute repeated in subclasses; types are correct.

6. Tutorials 2 and 2.1 solved

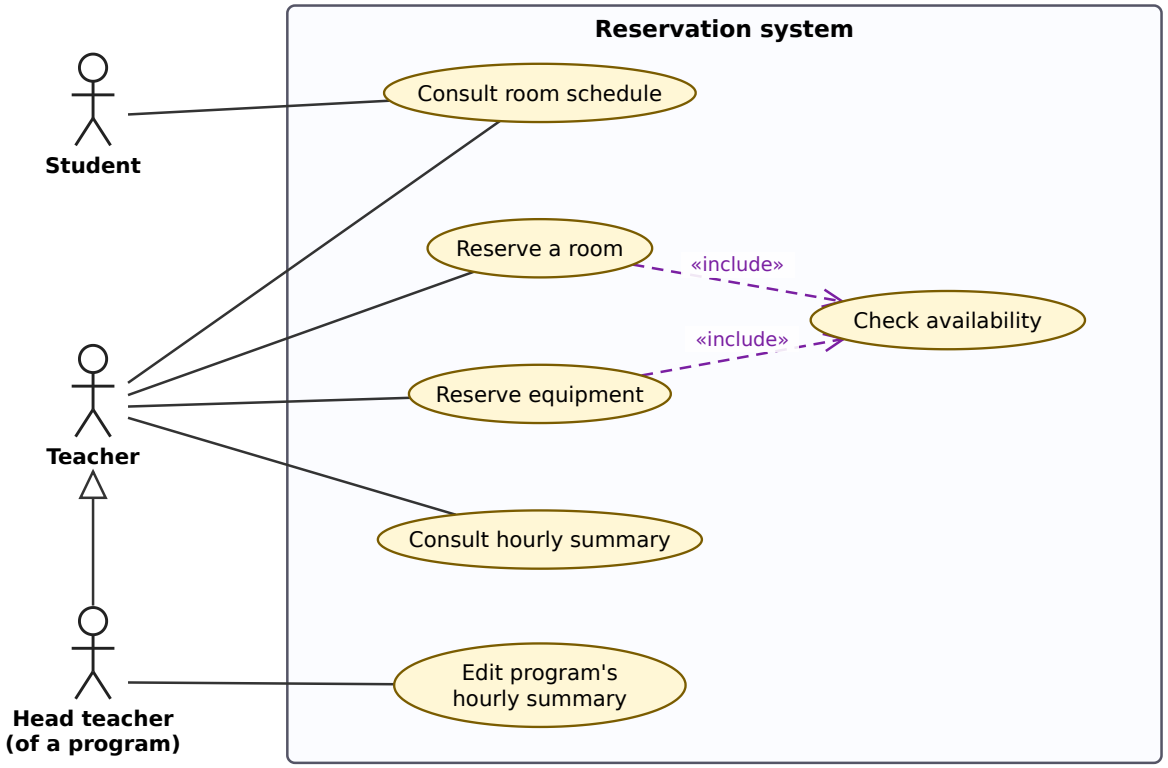
6.1 Tutorial 2.1: classroom and equipment reservation

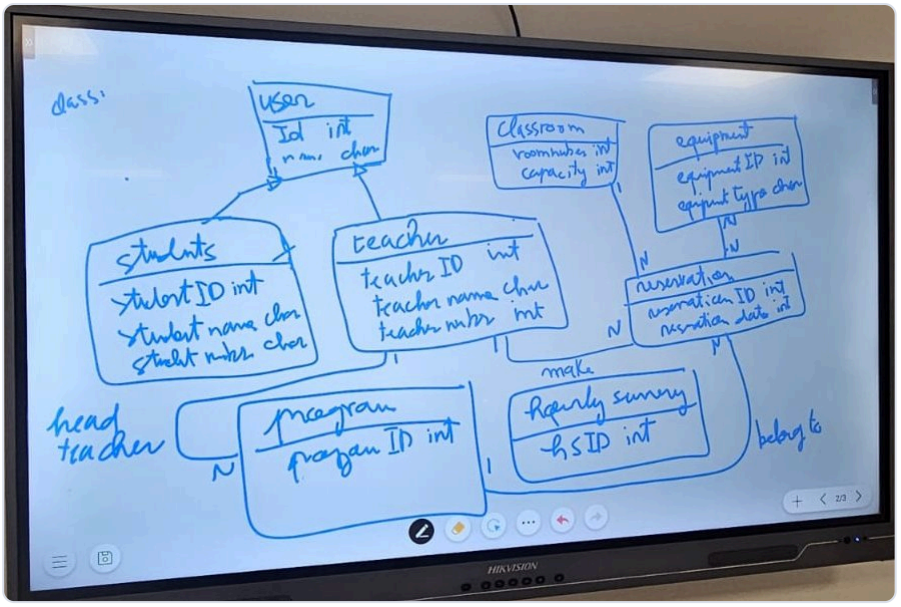


(board) use case diagram

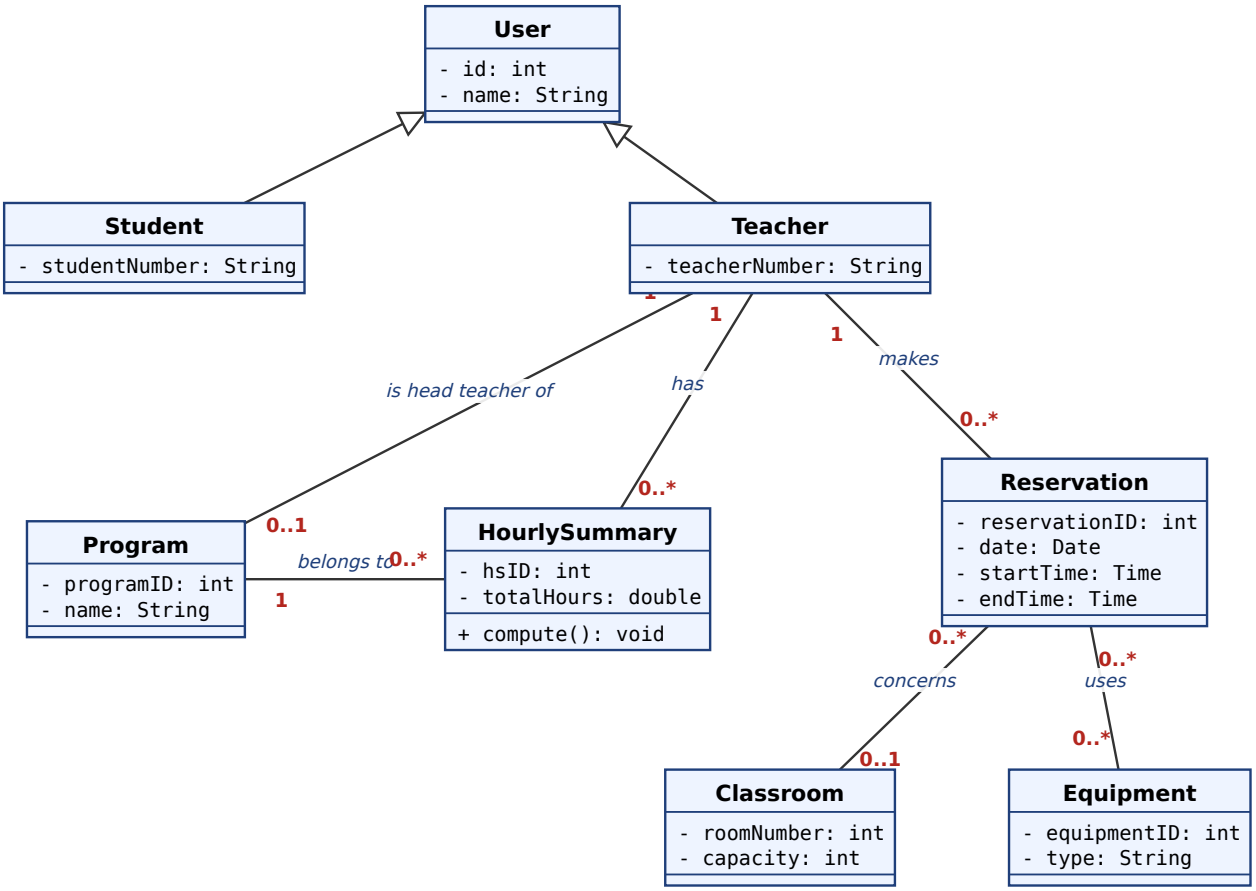
Reading the text like the prof:

- "Only teachers can make reservations" → Teacher: Reserve room, Reserve equipment.
- "subject to the availability" → both `«include»` Check availability.
- "room schedule can be consulted by everyone (teachers and students)" → Student and Teacher both linked to Consult room schedule.
- "hourly summary only accessible by teachers" → Teacher: Consult hourly summary.
- "a designated teacher who alone can edit the hourly summary of the program" → **Head teacher —▷ Teacher** + Edit program.





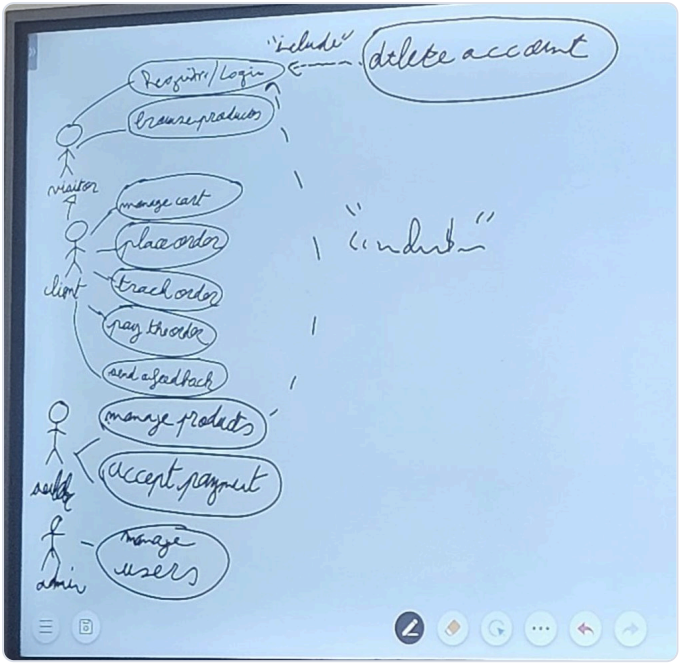
(board) class diagram drawn in class



What I changed from the board, and why (say it if asked)

- **Inherited attributes aren't repeated**: User already has id and name, so Student/Teacher don't repeat "student ID, student name" (that's the point of inheritance).
- Types: `char` → `String` for names/types; `reservationDate: int` → `Date`.
- A reservation can be for a room **and/or** equipment → Classroom `0..1`, Equipment `0..*` (the text says "classrooms as well as equipment (laptops and/or video projectors)"). Bonus: Laptop and VideoProjector can be subclasses of Equipment.
- "head teacher" is kept as an **association** Teacher → Program (like the board), with multiplicities in UML style instead of 1/N.

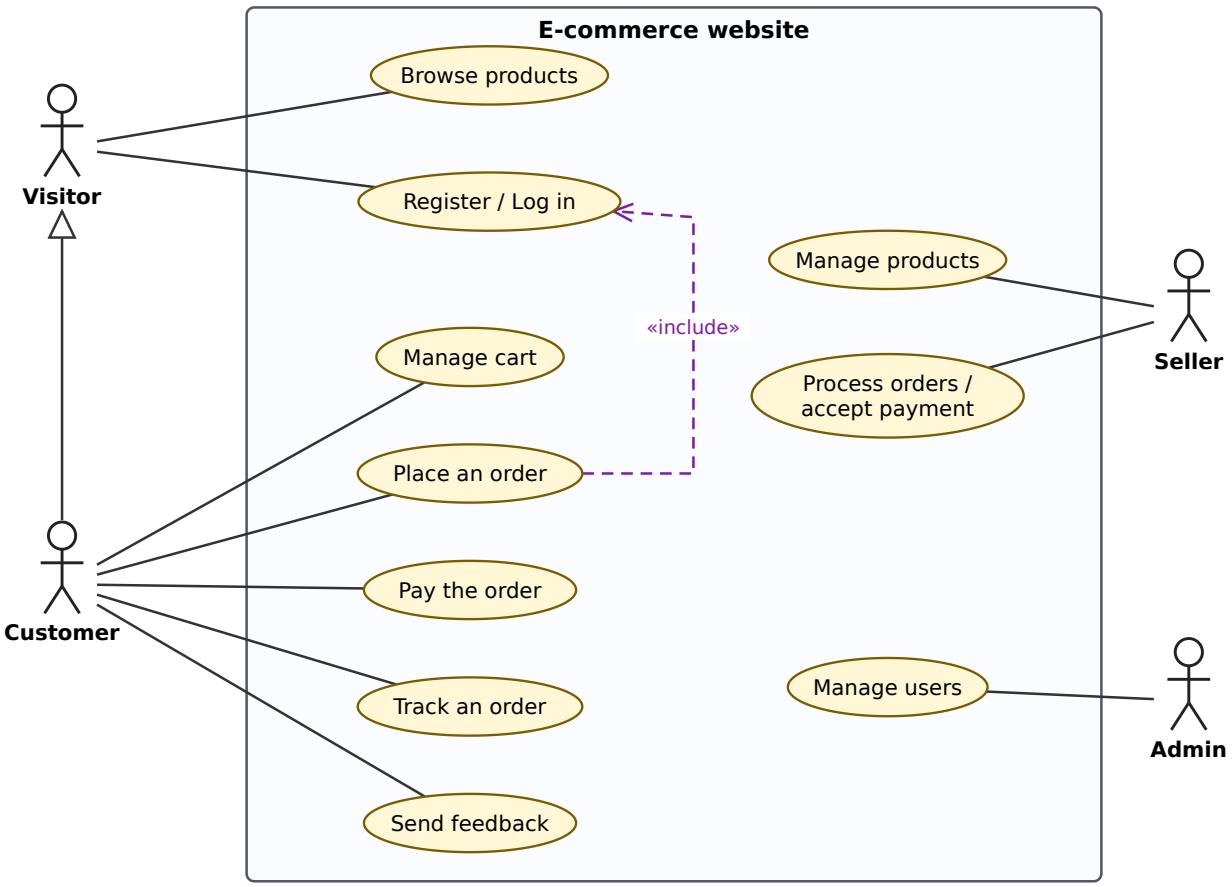
6.2 Tutorial 2 Ex1: e-commerce website (use case + class)

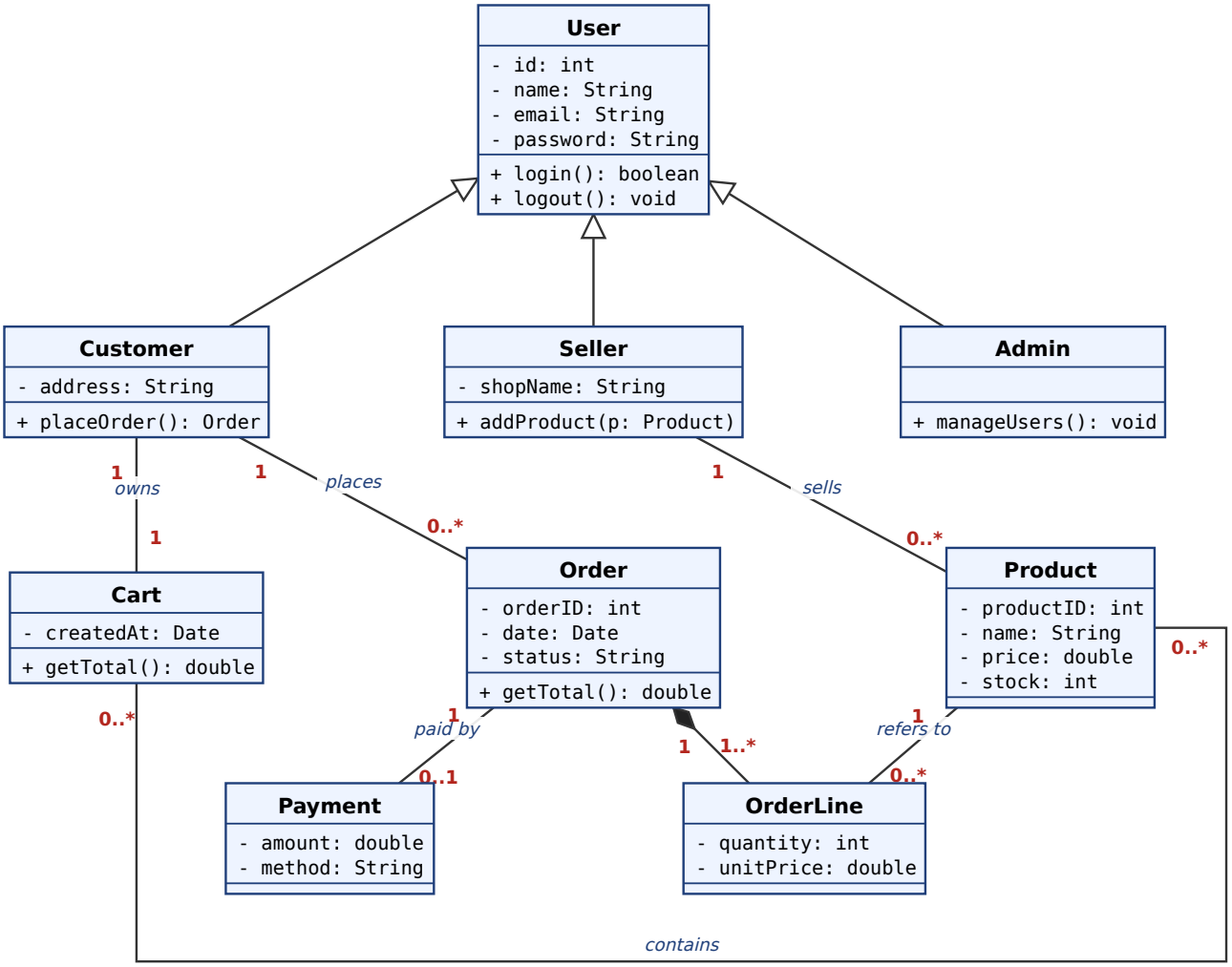


(board) use case diagram

The prof's actors: Visitor (register/login, browse products), Client —▷ Visitor (manage cart, place order, track order, pay the order, send feedback), Seller (manage products, accept payment), Admin (manage users). "Delete account" `«include»` Register/Login, and the client's operations need the login (dashed `«include»`).

The text only says "buy and sell physical goods, services and digital products, process orders, accept payments, manage shipping and logistics, customer service": the actors and UCs come from what such a site obviously needs. This kind of "open" text is where you get marks for **justifying** your actors.

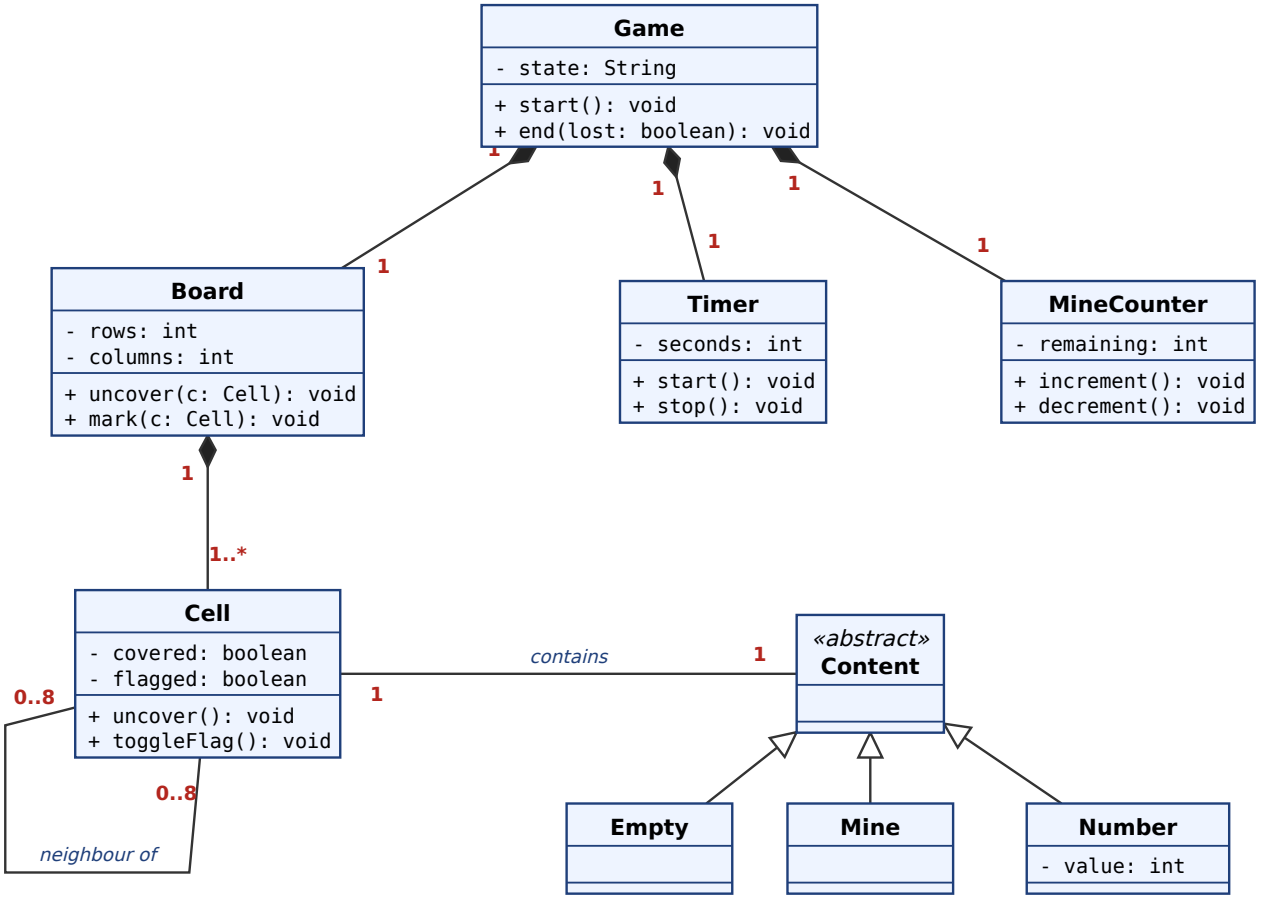

E-commerce website
Browse products
Register / Log in
Manage cart
Place an order
Pay the order
Track an order
Send feedback
Manage products
Process orders / accept payment
Manage users
«include»
Visitor
Customer
Seller
Admin


User
- id: int
- name: String
- email: String
- password: String
+ login(): boolean
+ logout(): void
Customer
- address: String
+ placeOrder(): Order
Seller
- shopName: String
+ addProduct(p: Product)
Admin
+ manageUsers(): void
1
owns
1
1
places
0..*
1
sells
0..*
Cart
- createdAt: Date
+ getTotal(): double
Order
- orderID: int
- date: Date
- status: String
+ getTotal(): double
Product
- productID: int
- name: String
- price: double
- stock: int
0..*
0..*
1
paid by
0..1
1
1..*
1
refers to
0..*
Payment
- amount: double
- method: String
OrderLine
- quantity: int
- unitPrice: double
contains

Composition Order ◆— OrderLine: an order line can't exist without its order. Cart — Product is many-to-many (a quantity could become an association class "CartItem"). Shipment and Feedback classes can be added the same way (Order 1 — 0..1 Shipment; Customer 1 — 0..* Feedback 0..* — 1 Product).

6.3 Tutorial 2 Ex2: Minesweeper (class diagram)

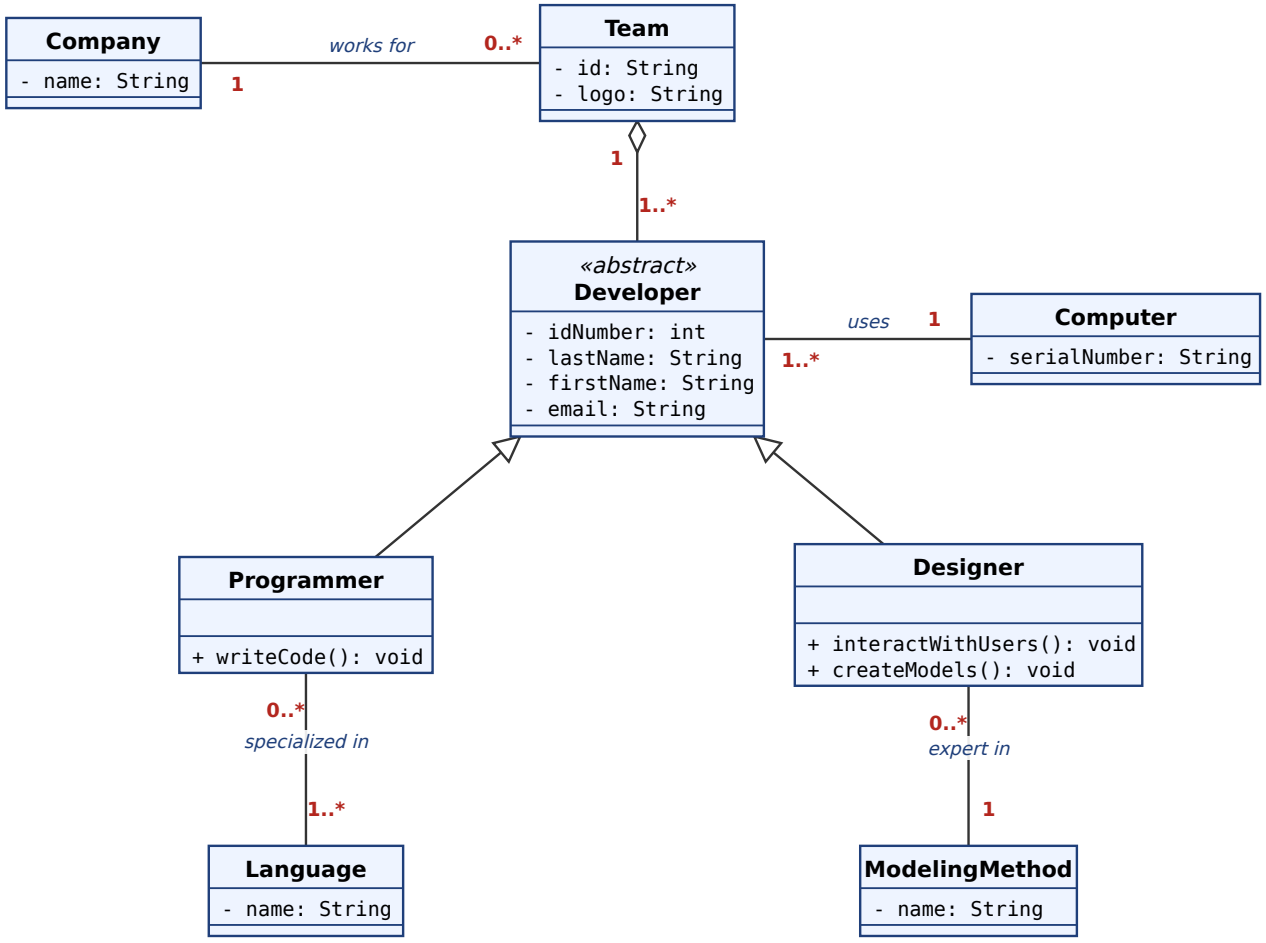
- "composed of a rectangular board, a timer and a mine counter" → **Game ◆ Board, Timer, MineCounter** (composition: they don't exist without the game).
- "The board is a grid of cells" → **Board ◆ 1..* Cell**; rows/columns are attributes of Board.
- "covered", "flagged" → boolean attributes of Cell; "uncover", "mark" → operations.
- "neighbouring cells" → a **reflexive** association Cell — Cell (0..8 neighbours).
- "The content can be: nothing, a mine, or a number" → abstract class **Content** with 3 subclasses (generalization); `value` only exists in Number.
- "The timer counts the seconds" → Timer.seconds; "counter shows mines remaining" → MineCounter.remaining with increment()/decrement() (the flag rules).
- The scenarios (blank → reveal neighbours, mine → lose) are **behaviour**: they become the operations uncover()/end(lost) (the details belong to dynamic diagrams).



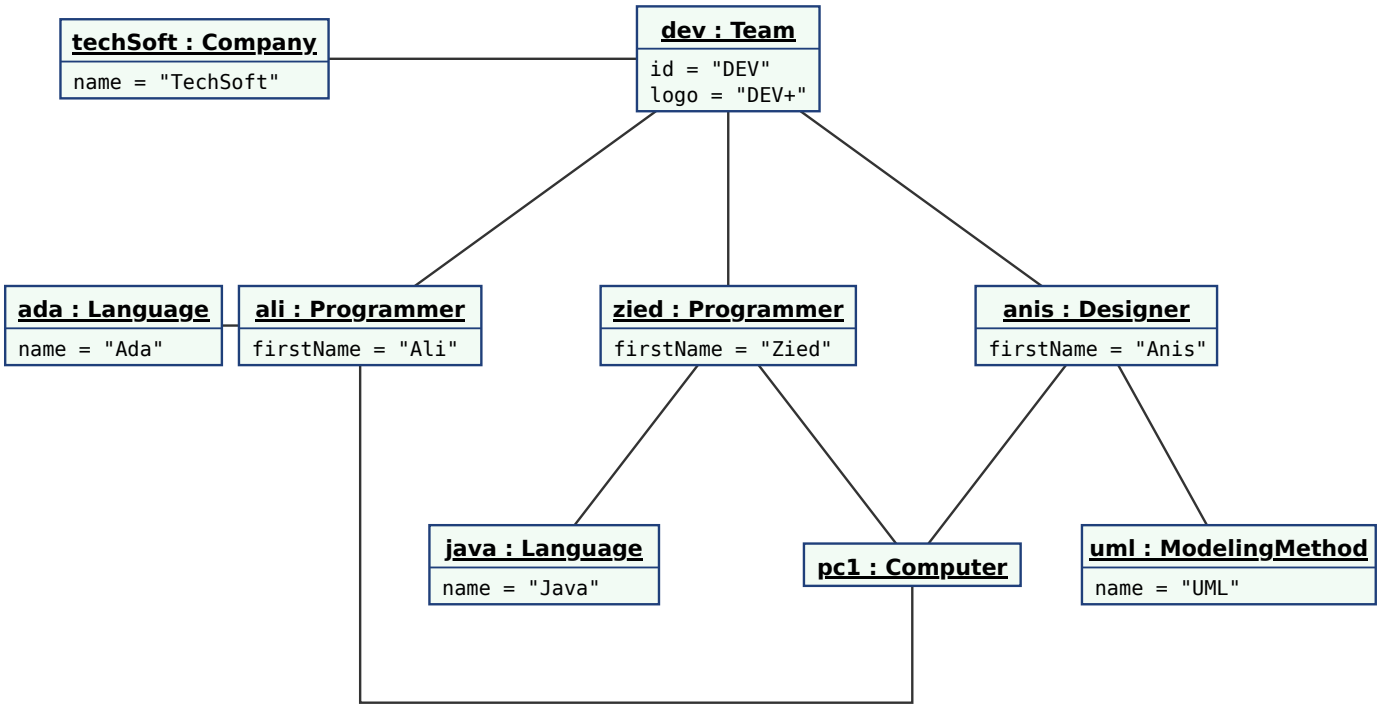
6.4 Tutorial 2 Ex3: development team (class + object diagram)

- "team composed of developers" → Team ◇ 1..* Developer (aggregation: a developer exists without the team).
- "works for a company", "identifier and a logo" → association to Company; id, logo attributes.
- "ID number, last name, first name, email" → attributes of Developer.
- "specialized in one or more languages" → Programmer — **1..*** Language; "expert in a modeling method" → Designer — **1** ModelingMethod.
- "write code" / "interact with users and create models" → operations.

- "can be a programmer… or a designer…" → **generalization**; Developer is abstract (every developer is one of the two).
- ⚠ "Each developer uses a personal computer" but Q2 says "**All developers use the same computer**" → the multiplicity must allow it: Developer **1..*** — **1** Computer (not 1 — 1). Exam trap: the object diagram must respect the class diagram!



Q2: object diagram ("Ali and Zied are programmers in Ada and Java, Anis is a UML designer, team DEV with logo DEV+ for TechSoft, all use the same computer"):



7. Exam-day checklist

Before drawing (5 min per exercise)

- Underline actors / circle verbs / box key words (preceded by, optionally, can be, all, only, composed of).
- Write the actor → UC table on the draft.
- For a class diagram: list classes, attributes and verbs between classes.

While drawing

- Pencil first, big rectangle, system name.
- Every dashed arrow labelled `«include»` / `«extend»`, in the right direction.
- Generalization = hollow triangle towards the parent.
- Class diagram: multiplicities on **both** ends of every association.

Final check (2 min)

- Is every sentence of the text covered by something in the diagram?
- No arrows between UCs without a stereotype? No flowchart?
- Actors outside, UCs inside, names = verb + complement?
- No attribute repeated in a subclass; types `String` / `int` / `double` / `Date` / `boolean`?
- Object diagram: underlined name : Class, values, no multiplicities, consistent with the class diagram?

Write 1–2 lines to justify your choices

e.g. "Engineer specializes Employee because engineers can also consult products". Teachers give marks for correct reasoning even if the drawing differs from theirs: in UML there's often more than one correct answer.